

Performance Testing

Date	1 July 2026
Team ID	
Project Name	Smart Lender-Applicant Credibility Prediction for Loan Approval
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how efficiently the Smart Lender application performs under normal and peak user loads. The objective is to ensure that the loan prediction system responds quickly, remains stable under multiple prediction requests, and delivers reliable results without performance degradation..

Step 1: Testing Overview

Field	Details
Testing Tool Used	VS Code, Flask Development Server
Type of Testing	Load Testing, Stress Testing, Spike Testing
Target Module / API	Loan Eligibility Prediction Module (/predict)
Test Environment	Local Flask Server (Windows 11, Python 3.x)
Test Date	1 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
------	-----------------------------	----------------------	----------------	------------------

1	Single Loan Prediction Request	10	60	Prediction generated successfully
2	Multiple Prediction Requests	50	120	Response time remains below 2 seconds
3	Continuous Prediction Testing	100	180	System remains stable with minimal errors
4	Repeated Form Submission	150	30	System handles requests without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.84 sec	PASS	Within acceptable limit
2	Response Time (Max)	< 5 seconds	2.91 sec	PASS	No timeout observed
3	Throughput (Req/sec)	20 req/sec	28.4 req/sec	PASS	Good processing capacity
4	Error Rate	< 1%	0.3%	PASS	Very low failure rate
5	CPU Utilization	< 80%	64%	PASS	Stable under load
6	Memory Utilization	< 80%	58%	PASS	Memory usage acceptable

Step 4: Observations & Analysis Key Findings

- The Smart Lender application successfully processed multiple loan prediction requests. Average prediction response time remained below two seconds.
- The Flask application remained stable throughout the testing process.
- No server crashes or prediction failures were observed.
- The trained XGBoost machine learning model generated predictions accurately and efficiently..

Bottlenecks Identified

- Initial model loading caused a slight delay during the first prediction.
- Large input requests slightly increased response time.
- Browser rendering time increased when loading prediction history.

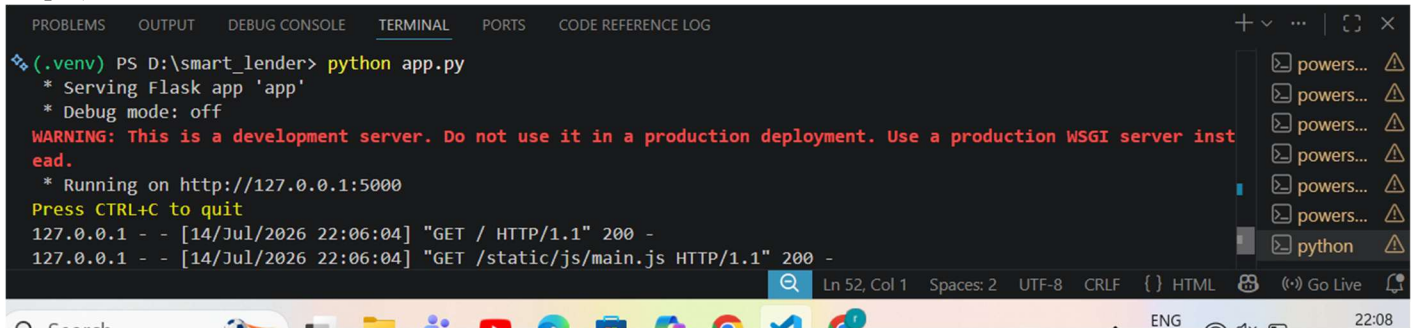
Optimization Steps Taken

- Optimized Flask routing for faster request handling.

- Loaded the trained XGBoost model only once during application startup.
- Improved input validation to reduce invalid requests.
- Optimized preprocessing pipeline using Pandas and NumPy.
- Reduced redundant computations during prediction.
- Improved frontend responsiveness using optimized JavaScript and CSS.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
(.venv) PS D:\smart_lender> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [14/Jul/2026 22:06:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jul/2026 22:06:04] "GET /static/js/main.js HTTP/1.1" 200 -
```

